

Comparing RNA models for something something

Bryan Richlinski *University of Kansas*

brichli@ku.edu Dominic Tassio *University of Kansas*

itsamemario@ku.edu Joseph Joestar *University of Kansas*

bobsyouruncle@ku.edu

Abstract

results indicate the entropy-ordered enumeration is a promising general-purpose technique to accelerate goal-directed top-down search for program synthesis.

I. INTRODUCTION

RNA molecules can be grouped into families based on shared characteristics such as structural patterns, sequence similarities, and functional roles. This allows researchers to study them in a more organized and meaningful way. To capture these similarities, a technique known as Multiple Read Alignment (MRA) is often employed, in which several RNA sequences from the same family are aligned together. From this alignment, a model can be constructed that encodes the common features of the family. This model can then be used as a reference to evaluate new RNA reads, providing a measure of how likely it is that a given sequence belongs to the same RNA family.

The chosen model for representing RNA families has often been a Stochastic Markov Model (SMM), which is capable of describing many RNA molecules in terms of both their raw symbolic strings and certain structural configurations that the molecules can take. This model has the same expressive power as a Stochastic Regular Grammar (SRG), meaning it can capture a wide range of sequence and structural patterns. However, it is limited in that it cannot represent more complex structures. These more intricate configurations are referred to as Secondary Structures (SS), and they require more powerful modeling techniques to be fully captured. To address this issue, a Covariance Model (CM) is often used to model these structures. The CM is a form of restricted Stochastic Context Free Grammar (SCFG) which captures a subset of the language of nested strings, the same set represented by Stochastic Visibly Pushdown Automaton (SVPA).

As we are primarily concerned with secondary structural properties, we choose to use the latter technique when comparing different RNA families. We hypothesize that RNA families themselves can be examined for similarity, by comparing the similarity of the models used to represent them. We propose a modified tree edit distance for examining the similarity of different Covariance Models which compares both structure and log-odds transitions.

We validate our algorithm as finding the smallest edit distance between two Covariance Models in Section II. To validate the efficacy of our approach we compare our results to the clan of RNA families. These clans encode how similar families are to each other based upon empirical evidence. By comparing the distance between models, to their respective clan we infer a cutoff and validate upon a hold out set. We evaluate this approach in Section V.

II. FORMALISM

A. Zhang-Shasha Algorithm

B. Hidden Markov Model Distance

C. Hidden Markov Model Distance

D. Hidden Markov Model

A Hidden Markov Model (HMM) consists of a finite set of states X and an alphabet Y . Each state $x \in X$ has an emission probability function, given as a vector $p(y|x_i)$, that gives the probability of emitting a symbol $y \in Y$, and each state x has a transition probability function that specifies the probability of moving from the current state to any other state in X at the next observation period, also given as a vector $p(x_t|x_{t-1})$.

This model can emit a sequence of observations, $y_{1:T}$. The likelihood that a given model has produced a particular observation sequence can be computed efficiently using the forward algorithm.

A naive solution to find the probability of being in state x at time T and emitting symbol y_T , by summing the probability of all possible paths which could have emitted the entire sequence and ended in state x . Instead the forward algorithm uses the markov property (the current state only depends on the previous state) to construct an optimal substructure. We can say that $(x_T|y_{1:T})$ only depends upon the solutions to $(x_{T-1}|y_{1:T-1})$, and the constant transition / emission probabilities applied to these subproblem solutions.

In algorithm 1, we present the forward algorithm. After initializing the model parameters and prior probabilities, we iterate through each time step t from 1 to T . We then update the probability of being in a given state by summing over all probabilities from the previous time step, weighted by their associated transition probabilities, and multiplying the sum by the emission probability of the current observation.

The final result of this step is the probability of being in each state at time T , given the entire sequence of observations up to that point. To calculate the total probability of the HMM emitting the sequence we sum over all final state probabilities.

Algorithm 1 Forward Algorithm for HMM

```

1: Initialize
2:    $t = 0$ 
3:   transition probabilities:  $p(x_t | x_{t-1})$ 
4:   emission probabilities:  $p(y_t | x_t)$ 
5:   observations:  $y_{1:T}$ 
6:   prior probability:  $\alpha(x_0)$ 
7: For  $t = 1$  to  $T$ :
8:    $\alpha(x_t) = p(y_t, x_t) \sum_{x_{t-1}} p(x_t | x_{t-1}) \alpha(x_{t-1})$ 
9: Return:  $p(y_{1:T}) = \sum_{x_T} \alpha(x_T)$ 

```

54 E. Distance Calculation

55 To compare two Hidden Markov Models H_1 and H_2 , we generate several sequences of observations from each model. Let
56 these sequences be labeled $h_{11}, h_{12}, \dots, h_{1n}$ from H_1 and $h_{21}, h_{22}, \dots, h_{2n}$ from H_2 . We approximate the Kullback–Leibler
57 (KL) divergence in both directions and then average over the number of sequences. While many other divergence measures could
58 be employed, such as the Jeffreys divergence, due to time constraints we only tested using KL divergence. The approximation
59 is given by

$$D(H_1, H_2) \approx \frac{1}{2n} \left(\sum_{i=1}^n \log \frac{p(h_{1i} | H_2)}{p(h_{1i} | H_1)} + \sum_{i=1}^n \log \frac{p(h_{2i} | H_1)}{p(h_{2i} | H_2)} \right).$$

60 Here $p(h | H)$ denotes the probability of sequence h under model H , computed using the forward algorithm.

61 III. ANALYSIS

62 A. ZSS Complexity

63 B. HMM Complexity

64 The forward algorithm has time complexity $O(nm^2)$, where n is the length of the observation sequence and m is the number
65 of hidden states. At each time step, the algorithm must compute the probability of transitioning from all states at time $t - 1$ to
66 all states at time t :

$$\alpha(x_t) = p(y_t, x_t) \sum_{x_{t-1}} p(x_t | x_{t-1}) \alpha(x_{t-1}).$$

67 The emission probability $p(y_t | x_t)$ acts only as a multiplicative constant in this computation, so the dominant cost arises
68 from the double summation over states, giving m^2 . Since this computation is performed for each of the n time steps, the
69 overall time complexity is $O(nm^2)$.

70 The label distance algorithm requires we use the HMM to generate many random, but statistically likely, sequences, each of
71 indeterminate length. As this is a random process, a worst-case time complexity cannot be specified without fixing the length of
72 the generated strings. However, if the length of a string is known, the complexity of its generation is linear in that length. Thus,
73 the cost of string generation can be folded into the overall complexity of the forward algorithm, yielding the same $O(nm^2)$
74 bound when combined with the dynamic programming steps.

75 IV. IMPLEMENTATION

76 The algorithm is implemented in Python. We make use of the `hmmlearn` library to store Hidden Markov Models, to emit
77 sequences of observations, and to apply the forward algorithm for likelihood calculations. For the ZSS algorithm, we employ
78 the `zss` package to perform the Zhang–Shasha tree edit distance, augmented with our custom distance metric to tailor the
79 comparison to our models.

80 A. Node HMM Model Augmentations

81 In contrast to the formal description of the algorithm, the concrete implementation requires structural adjustments to the
82 Hidden Markov Models. Specifically, each model is augmented with a single entry state and a final exit state. The reason for
83 the initial incoming transition is that, while we represent each node as an HMM, this is a simplification, a node may have
84 several incoming and several outgoing transitions. To account for this, these incoming and outgoing transitions are collected
85 and normalized into one incoming and one outgoing state, with transition probabilities summed and normalized. This ensures
86 that every sequence begins from a unique entry point and terminates in a unique exit point.

87 As well, in the concrete models there are hidden states that are silent. The forward algorithm cannot directly handle the
88 potentially infinite paths that arise from such ϵ -transitions, so these states must be removed. Eliminating silent states while
89 preserving the transition and emission probabilities of the non-silent states requires an $O(n^3)$ preprocessing step, where n is
90 the number of states in the model. This transformation is performed once for each node.

91 Alternatively, we could have chosen to force every silent state to emit a self-identifying character. We believe this modification
92 would shift the comparison between HMMs toward a more structural rather than emission-based distance measure. Further
93 research could evaluate the effect of this approach.

94 Because the ZSS algorithm must also account for insertions and deletions, we require a consistent rule for comparing a node
95 to its absence. We handle this by comparing the observed sequences emitted from the node, to the probability that it emits the
96 empty string.

$$D(H, \emptyset) \approx \frac{1}{2n} \left(\sum_{i=1}^n \log \frac{p(h_i | H)}{p(\emptyset | H)} + \sum_{i=1}^n \log \frac{p(\emptyset | H)}{p(h_i | H)} \right).$$

97 Here $\emptyset$ denotes the empty string, and h_i are the observed strings generated from the model H .

98 Additionally, certain nodes in the concrete model cannot be represented internally as Hidden Markov Models. Specifically,
99 the BIF and END nodes are treated as structural markers rather than probabilistic models. Any time these nodes are compared
100 to like named nodes they will return a log-odds of 0.0 (perfect match) and a default high value otherwise. For this test we
101 chose -5.0 odds.

102 B. Memoization

103 To reduce redundant computation, many values are memoized during parsing and model construction. All sample strings
104 required for label comparison are generated once and stored, together with the probability that the generating model produced
105 it, given by the forward algorithm.

106 Furthermore, when comparing two CM's with the ZSS algorithm, the distance between nodes is memoized after the first
107 computation. Subsequent distance measures are retrieved directly from this cache, eliminating repeated evaluations and improving
108 overall efficiency.

109 V. EVALUATION

110 Having implemented this approach for comparing covariance models, we need a way to evaluate how well it performs. We
111 explore two methods to evaluate how well our approach compares covariance models. First, is an extremely naïve statistical
112 analysis that will use the RNA family's identified clan to compare within- and between-group similarity. Second, is a more
113 nuanced approach that utilizes concrete sequence comparison as a base truth for comparing the models.

114 Due to time constraints, we only fully implemented the first comparison method. We will still describe the second method as
115 it is useful for context and potential future work.

116 For this first method, we utilized the clan classification given to some groups of RNA families. Naïvely, we assume that
117 the distance of covariance models within a clan is on average smaller than the distance of covariance models between clans.
118 However, using clans to perform within- and between-group analysis is flawed for several reasons. Primarily, the clans are
119 not strictly organized based on structure. As our approach implements a structural comparison between covariance models,
120 comparing the distance of models within clans and models between clans will not necessarily be representative. Another flaw
121 being the variability in the number of identified RNA families in a clan. As we have an uneven distribution of potential within-
122 and between-clan comparisons, our comparison will need to be resilient to that bias in the dataset. Despite these flaws, we still
123 attempt this comparison method.

124 Permutational multivariate analysis of variance (PERMANOVA) is a good statistical test for this comparison. Compared to
125 ANOVA, which compares the means of groups, PERMANOVA uses a distance measurement between values. This matches
126 perfectly with our need to evaluate our distance comparison and mitigates the impact of our disparate group sizes.

127 We arbitrarily selected 40 covariance models across 10 clans for use in our evaluation. This gives 780 combinations of
128 families to compare. On an M4 MacBook Pro with 24GB of RAM, this took approximately 3 hours to complete. Given the
129 distance from these combinations, we are able to construct the distance matrix necessary for PERMANOVA. Running this

statistical test resulted in a test statistic of **1.029482** and a p-value of approximately **0.4**. The test statistic describes the similarity of the within- and between-group comparisons. We will discuss the implications of these results in Section VI.

Our second method of evaluation is to use the comparison of RNA sequences belonging to an RNA family as a baseline truth to measure our comparison of covariance models. As the comparison of RNA sequences is a structural comparison, the flaw of using clans would not be an issue. If our method of comparing covariance models is an effective measurement, we would expect there to be a correlation for the distance measured between RNA sequences of different families and distance between covariance models. Unfortunately, due to time constraints and our inexperience in bioinformatics, we were unable to perform this analysis.

VI. DISCUSSION

From our evaluation of our comparison approach using PERMANOVA, we have found that using the Zhang-Shasha tree edit distance algorithm and our HMM distance cost function did not produce statistically significant results. Due to our finding of a p-value of approximately **0.4**, we can conclude that our approach is likely due to random chance. Further, the test statistic of **1.029482** means that there is effectively no difference in the within- and between-group distances.

There are likely several reasons why this is the case. Most importantly, the flaw in using RNA clans as the category in the PERMANOVA test as previously mentioned. Due to clans not being a purely structural classification, using it in the PERMANOVA test was likely meaningless.

Another potential reason is our small dataset. We only considered 40 covariance models and 10 clans. At the time, there are 4,170 RNA families and 147 clans on Rfam, meaning that there is a much larger dataset we could reach for.

Using all 4,170 RNA families (assuming each family on Rfam has a covariance model) would mean just under 8.7 million comparisons to make. It is unlikely we could have completed our approach on the entire dataset in a reasonable time frame using the consumer hardware available to us. Despite this limitation, it is possible that the PERMANOVA test on this larger dataset could produce statistically significant results. However, due to the previously discussed issue with using clans as a category, this approach would likely still be flawed.

The second method of using correlation of RNA sequences comparison to covariance model comparison is a much more promising avenue for fairly evaluating our approach. RNA sequence comparison or alignment is a well studied problem in bioinformatics, with a number of approaches and techniques. Given enough time, we likely could have repurposed an existing tool for this. We regret that we were unable to conduct this evaluation in time to include in this paper.

One benefit of our approach is the actual run time for comparing two models. While we have no concrete data for any one particular run, being able to perform 780 comparisons in approximately 3 hours on consumer hardware represents a reasonable run time. Due to our implementation being written in Python and utilizing no amount of parallelization or caching between comparisons, it is likely that there are significant performance gains to make. As each comparison is independent of any other comparison, parallelization would be an easy optimization that would likely see time improvements, even on consumer hardware.

There are threats to our approach. While Zhang-Shasha is a competitive algorithm for tree edit distance of ordered and labeled trees, it is possible that our chosen ordering of nodes from the covariance model negatively impacted our results. It is also possible that the cost function we implemented to determine the cost of deleting, inserting, and re-labeling nodes was not optimal. Our HMM fragment to HMM fragment is a probabilistic process involving produced sequence likely hood and could be potentially tuned for better or worse results. Without a rigorous method of evaluation, these individual aspects of our approach are difficult to properly evaluate and potentially tune.

Overall, we find our approach to still have potentially despite the negative results from the PERMANOVA analysis. Given more time and a better method of evaluation, we believe this approach could have more impactful results.

VII. RELATED WORK

VIII. CONCLUSION

We present our approach for structural comparison of RNA family covariance models using tree edit distance and HMM to HMM comparison. The Zhang-Shasha algorithm is a competitive and robust method of computing the distance between two ordered and labeled trees in a reasonable time complexity for the task. Our HMM to HMM comparison leverages the internal structure of nodes within a covariance model to produce a cost function that can be used in the tree edit distance algorithm. Together, we find these two approaches to be an interesting, if not promising, way to compare covariance models.

Despite the flaws in our evaluation of this approach, we believe that utilizing a different method of evaluation would allow us to better scrutinize this approach and present ways to improve the details of our HMM to HMM comparison. It is unfortunate that we were unable to perform a better evaluation before the writing of this paper. This leaves significant future work open for developing and evaluating this approach.

Ultimately, we find that this approach was a worthwhile first attempt at using tree edit distance for comparison of covariance models.

IX. DISCLAIMER

183

184 This report was written with assistance from an llm working in a strictly editorial capacity. All content and ideas are the
185 original work of the authors.